

Note technique - Segmentation des images

Sommaire

- 1 - Introduction
- 2 - Présentation de la segmentation d'images
- 3 - Approches
- 4 - Les données
- 5 - Générateur de données
- 6 - Augmentation de données
- 7 - Métriques et Fonction de perte
- 8 - Modélisation
- 9 - Résultats
- 10 - Conclusion
- 11 - Références

1 - Introduction

Dans le projet système embarqué de vision par ordinateur, la segmentation des images est une des quatre parties du projet.

Après acquisition et traitement des images, celles-ci sont segmentées puis utilisées par le système de décision.

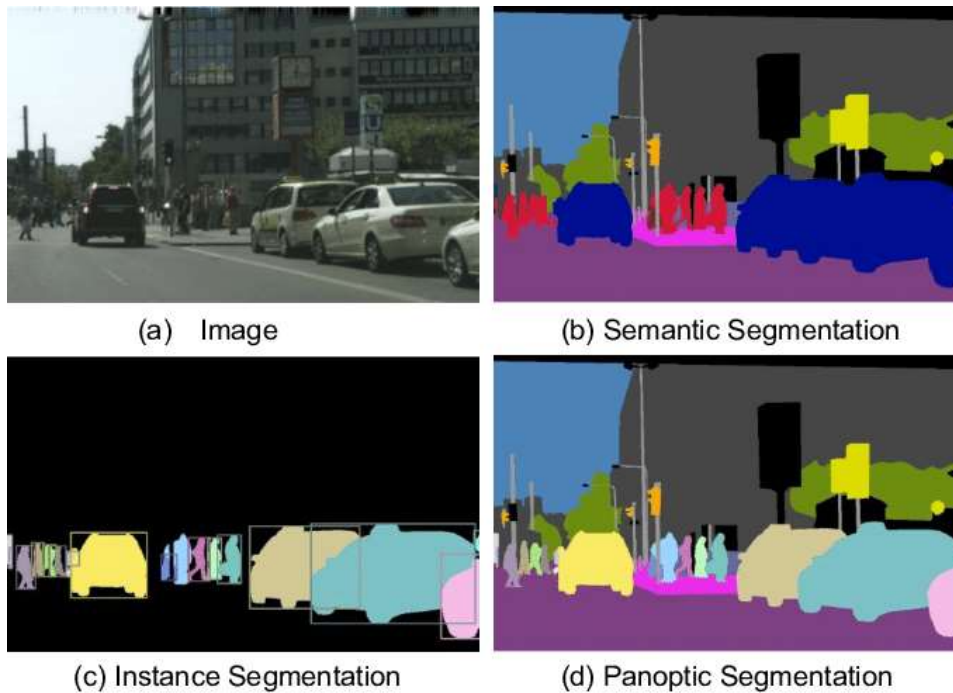
2 - Présentation de la segmentation d'images

La vision par ordinateur (computer vision) permet d'acquérir une compréhension des images et vidéos. La classification des images et la segmentation des images sont des tâches de la vision par ordinateur.

- La classification d'images permet de labelliser une image, de la classer dans une catégorie.
- La segmentation d'images permet d'identifier différents éléments sur une image. Chaque pixel est labellisé, classifié dans une classe relative à l'élément (exemple chien, chat...) à identifier et localiser.

On distingue trois types de segmentation d'images :

- La segmentation sémantique classifie chaque pixel sans distinguer deux instances d'un même élément.
- La segmentation d'instances différencie chaque instance en attribuant un label différent à chaque instance d'un même élément.
- La segmentation panoptique qui regroupe les deux techniques précédentes.



Domaines d'application

La segmentation sémantique est utilisée dans différents domaines comme les véhicules autonomes, l'imagerie médicale ou l'analyse d'images satellites par exemple.

3 - Approches

Différents algorithmes ont été développés pour la segmentation d'image.

La segmentation par approche régions, par approche frontières, par classification ou seuillage.

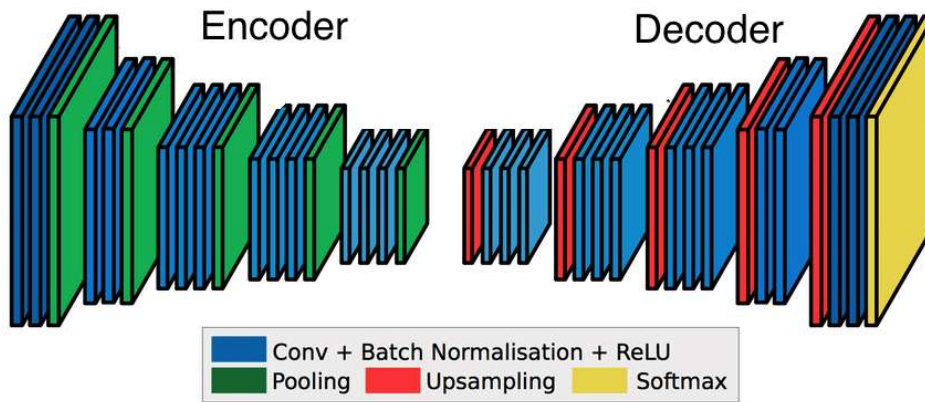
Aujourd'hui le deep learning est très utilisé, apportant de meilleures performances que les méthodes traditionnelles.

Pour la segmentation (et la classification) des images, les réseaux de neurones sont basés sur des CNN (convolutional neural network). Les CNN permettent d'extraire les caractéristiques d'une image.

Pour la classification, après cette première partie, s'ensuit une deuxième partie « fully connected » afin de classer l'image.

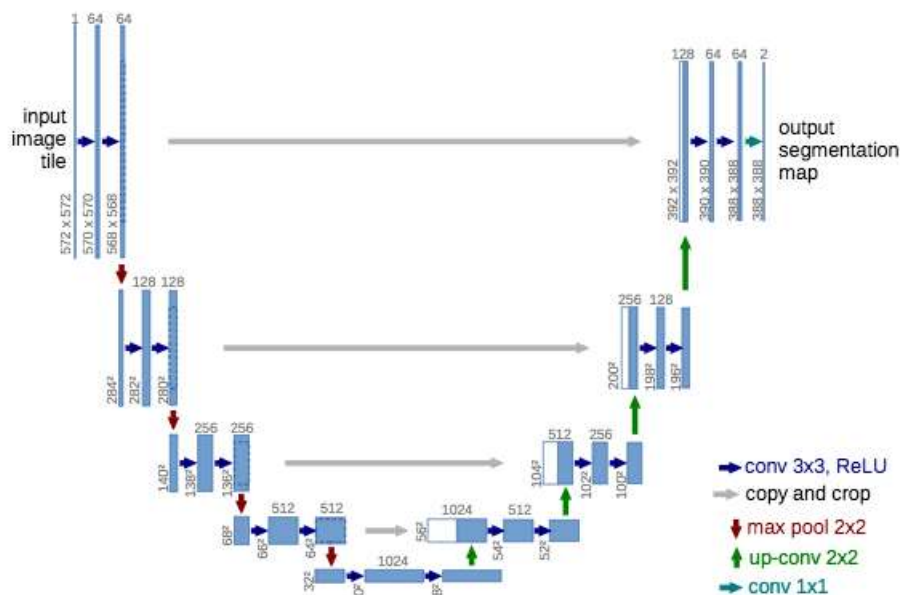
Pour la segmentation des images, au lieu d'effectuer une classification de l'image, la sortie est de même résolution que l'image en entrée et chaque pixel est classifié. Cette partie doit donc reconstruire l'image (upsampling, localisations...).

Cette partie est appelée décodeur et la première partie d'extraction de caractéristiques est appelée encodeur.



Un des modèles les plus populaires basé sur cette architecture est Unet.
Unet a été initialement conçu pour la segmentation d'images médicales en 2015.

L'architecture Unet



Unet est une architecture encodeur/décodeur symétrique.
L'architecture utilise des « skip connections » entre couches correspondantes.
Les informations perdues lors de l'encodage sont récupérées par le décodeur par l'intermédiaire de ces connections afin de reconstruire l'image.

4 - Les données

Les données utilisées proviennent du dataset CityScape.
Le dataset contient 5000 images représentant des scènes urbaines de 27 villes allemandes.
Les images sont réparties dans des dossiers train / val / test.

Pour chaque image, le dataset contient aussi les fichiers color, labelsId, instanceId, polygons.

Pour la segmentation sémantique :

le fichier labelsId contient les labels de chaque pixel

le fichier color contient les valeurs RGB associées au label

Les labels représentent 34 classes réparties dans 8 catégories :

```
{'void': [0, 1, 2, 3, 4, 5, 6],  
  'flat': [7, 8, 9, 10],  
  'construction': [11, 12, 13, 14, 15, 16],  
  'object': [17, 18, 19, 20],  
  'nature': [21, 22],  
  'sky': [23],  
  'human': [24, 25],  
  'vehicle': [26, 27, 28, 29, 30, 31, 32, 33, -1]}
```

5 - Générateur de données

Le nombre de données est important.

La quantité de mémoire peut être vite insuffisante pour traiter ce volume de données.

Pour remédier à cela, une solution est de développer un générateur de données qui fournira les données par batchs.

Keras fournit une classe Sequence qui permet de créer son propre générateur de données en utilisant le multi processing.

Pour cela, il suffit de créer une classe héritant de la classe Sequence et d'implémenter au moins les méthodes `__len__` et `__getitem__`.

Le générateur de données prend en entrée :

- une liste de fichiers à charger
- la liste des fichiers labels associés
- la taille du nombre de fichiers à charger à chaque séquence (batch size)

Chargement des données

Pour entraîner les modèles de deep learning, il faut fournir au modèle les images et leurs masques associés. De plus, des données de validation seront utilisées.

Une fonction `load_data` a été développée permettant de retourner

- une liste d'images pour l'entraînement
- une liste de masques pour l'entraînement
- une liste d'images pour la validation
- une liste de masques pour la validation

La fonction prend en paramètre le nombre de données d'entraînement voulues et la proportion ou la taille des données de validation.

Afin d'utiliser des données réparties au mieux sur l'ensemble du jeu de données, la fonction répartit les données parmi les villes disponibles en tailles égales.

La fonction `load_data` permet de générer les listes de fichiers en entrée des générateurs de données qui seront utilisés pour l'entraînement des modèles.

6 - Data augmentation

Si le nombre de données est insuffisant, le modèle aura tendance à faire du sur apprentissage. Une des solutions est donc d'augmenter les données.

S'il n'y a pas de données disponibles, avec les images, une solution est d'effectuer une augmentation de données qui effectuera des opérations sur les images disponibles.

La data augmentation peut être utilisée de deux façons :

- «offline»: En amont de l'entraînement, pour créer de nouvelles images.
- «online»: Lors de l'entraînement, avec une génération au fur et à mesure.
Cette méthode est généralement utilisée avec un data generator.

L'augmentation permet non seulement d'avoir plus de données mais elle peut aussi permettre de varier les images, afin d'aider le modèle à pouvoir mieux généraliser et ainsi obtenir de meilleures performances avec de nouvelles images.

Dans ce projet, les ressources de calcul et mémoire disponibles étant limitées, le nombre d'images en entrée est aussi limité.

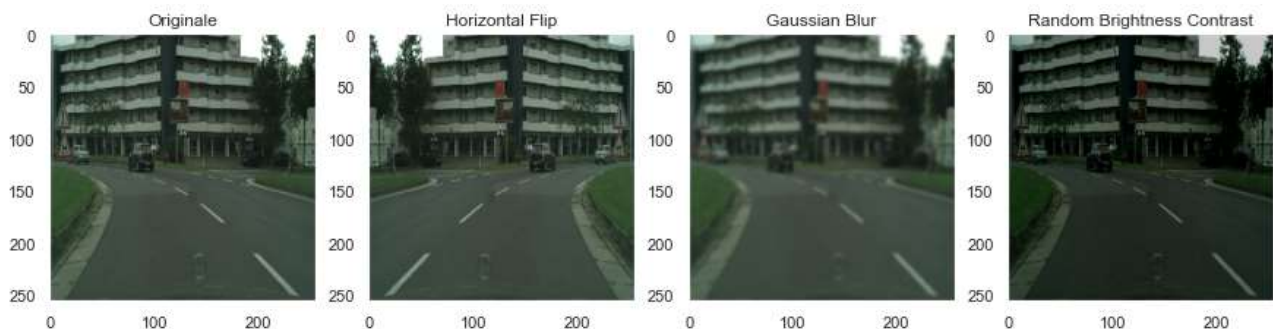
Au lieu d'augmenter le nombre d'images, l'augmentation de données est utilisée au sein du data generator pour remplacer l'image par une image augmentée (de manière aléatoire).

Plusieurs transformations ont été définies, et sont appliquées ou pas à l'image source et à son masque.

Pour ce faire, la bibliothèque `albumentations` a été utilisée.

3 transformations ont été utilisées :

- Horizontal Flip
- Gaussian Blur
- Random Brightness Contrast



7 – Métriques et fonctions de perte

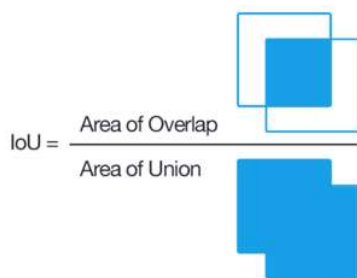
Métriques

La métrique accuracy n'est pas suffisante pour mesurer la performance du modèle.

Les images peuvent avoir une répartition déséquilibrée des classes, on peut donc obtenir une « accuracy » très bonne avec un modèle fournissant un seul label si celui-ci représente la majorité de l'image (image de fond par exemple).

Une métrique très utilisée en segmentation d'images est :
Intersection-Over-Union (IoU)

Cette métrique représente le rapport entre l'intersection et l'union de la prédiction attendue et celle effectuée.



Si toutes les prédictions sont identiques, alors l'intersection égale l'union, l'iou égale 1.
Si aucune prédiction n'est bonne, alors l'iou égale 0.

Une autre métrique utilisée est « dice coef ». Elle est proche de l'iou. Elle représente le rapport entre deux fois l'intersection divisé par le nombre total de pixels.

L'iou met plus l'accent sur les erreurs que dice coef. L'iou favorisera un modèle qui ne fera pas de grosses erreurs tandis que le dice coef favorisera un modèle qui globalement fait de bonnes prédictions.

Les fonctions de perte

Cross entropy est communément utilisée comme fonction de perte pour les problèmes de classification.

Categorical cross entropy permet d'évaluer l'erreur lors de classifications multi classe.

Cependant, pour les mêmes raisons qu'énoncé précédemment, lors de données avec des classes déséquilibrées, l'erreur de classification est mesurée pour chaque pixel, la classe dominante sera influente.

En général, pour la segmentation d'images, la fonction communément utilisée est « dice loss ». Elle correspond à $1 - \text{dice coef}$.

8 - La modélisation :

Pour effectuer la modélisation, la bibliothèque « segmentation models » a été utilisée.

https://github.com/qubvel/segmentation_models

Elle propose 4 architectures pour la segmentation d'images et 25 backbones.

- Architectures:

La bibliothèque propose les modèles Unet, Linknet, PSPNet et FPN.

Unet est une référence pour la segmentation d'images et sera utilisé en baseline.

Sur la présentation du projet Linknet:

« Linknet un réseau de neurone léger conçu pour la segmentation sémantique qui peut être utilisé pour des tâches comme les véhicules autonomes... »

La présentation de ce réseau décrit un réseau adapté à la problématique de ce projet. Il sera utilisé en complément de Unet.

- Backbones

La bibliothèque segmentation models propose une série de backbones pré-entraînés, correspondant à la partie encodeur du modèle.

Parmi la liste, le backbone VGG16 est un des modèles de référence.

Le modèle d'Oxford a gagné la compétition ImageNet en 2013. C'est un des premiers modèles de deep learning pour la classification d'images.

La bibliothèque propose aussi le modèle Resnet, plus récent. Le modèle de Microsoft a gagné la compétition ImageNet en 2016. Ce modèle a un nombre de couches plus grand. Il est aujourd'hui un standard.

VGG16 sera utilisé pour la baseline, et Resnet sera aussi testé.

- Modélisation

La baseline est donc constituée du réseau Unet et VGG16.

Ensuite, plusieurs essais ont été effectués avec :

- un autre backbone (resnet)
- une autre architecture que Unet (linknet)
- une autre fonction de perte
- avec l'augmentation de données

9 - Les résultat de l'étude

model	backbone	loss	epochs	sample	batchsize	augmentation	iou_score	fit_time	time_per_epoch
Unet	vgg16	crossentropy	12	200	16	no	44.46	209	17
Unet	resnet50	crossentropy	20	200	16	no	69.13	344	17
Linknet	resnet50	crossentropy	20	200	16	no	65.66	332	17
Unet	resnet50	dice_loss	20	200	16	no	76.53	350	18
Unet	resnet50	dice_loss	20	500	16	no	81.72	857	43
Unet	resnet50	dice_loss	20	500	16	yes	80.57	962	48

- 1) La baseline Unet / VGG16 / crossentropy a obtenu un iou score de 44,46.
Le nombre d'epochs s'est arrêté à 12 par un early-stopping.
Le score sur les données de validation n'augmentait plus, contrairement aux données d'entraînement. Nous sommes ici dans un cas d'overfitting.
- 2) Avec le backbone resnet50, le score est de 69,13, c'est celui-ci qui a été retenu.
- 3) Avec un réseau Linknet , le score est de 65,66, un peu inférieur au score précédent.
- 4) Unet/resnet50 a été conservé et entraîné avec une fonction de perte « dice loss ».
Le score a été clairement amélioré à 76,53.
- 5) Il est intéressant d'essayer d'entraîner ce modèle avec plus d'images.
Le score a été amélioré à 81,72.
- 6) Avec l'augmentation des données, le score est similaire, mais il est un peu inférieur à l'essai précédent. Pour que la data augmentation apporte un plus, il faudrait entraîner le modèle sur beaucoup plus d'images afin qu'il y ait de nombreuses images augmentées.

Le modèle (5) Unet/resnet50 entraîne sur 500 images avec « dice loss » a été retenu, avec un Iou score de 81,72.

10 – Conclusion

La modélisation a permis d'obtenir un iou score de plus de 80 %.
Ce résultat a été obtenu après avoir comparé plusieurs architectures de réseaux de neurones avec différents backbones.

Pour améliorer les performances, différentes pistes peuvent être étudiées :

- L'entraînement du modèle sur beaucoup plus d'images et plus d'epochs
Pour cela, il faut avoir des ressources de calcul conséquentes.
- La génération de nouvelles données à partir du jeu d'entraînement suffisamment important.
- Tester d'autres techniques d'augmentation de données
- Optimisation du batch size à la génération de données
- Tester d'autres architectures et backbones

11 – Références

https://fr.wikipedia.org/wiki/Segmentation_d%27image

<https://www.jeremyjordan.me/semantic-segmentation>

<https://stats.stackexchange.com/questions/273537/f1-dice-score-vs-iou>

<https://codeac29.github.io/projects/linknet/>

<https://towardsdatascience.com/understanding-semantic-segmentation-with-unet-6be4f42d4b47>

<https://divamgupta.com/image-segmentation/2019/06/06/deep-learning-semantic-segmentation-keras.html>